

Week 4 Assignment – The Full ML Workflow (Logistic Regression & kNN Classifiers)**Part I - Dataset 1:**

Figure 1 presents a two-dimensional visualisation of the training dataset as a scatter plot, where each point corresponds to a single observation consisting of two input features. The x-axis represents the first feature, and the y-axis represents the second. Points are coloured according to their target, where blue indicates the positive class (+1), and red represents the negative class (-1). A black edge outline is included to make each point clearly distinguishable. From this plot, it is evident that the two classes are not linearly separable; the positive and negative samples are heavily intermixed with no clear linear boundary capable of distinguishing between them.

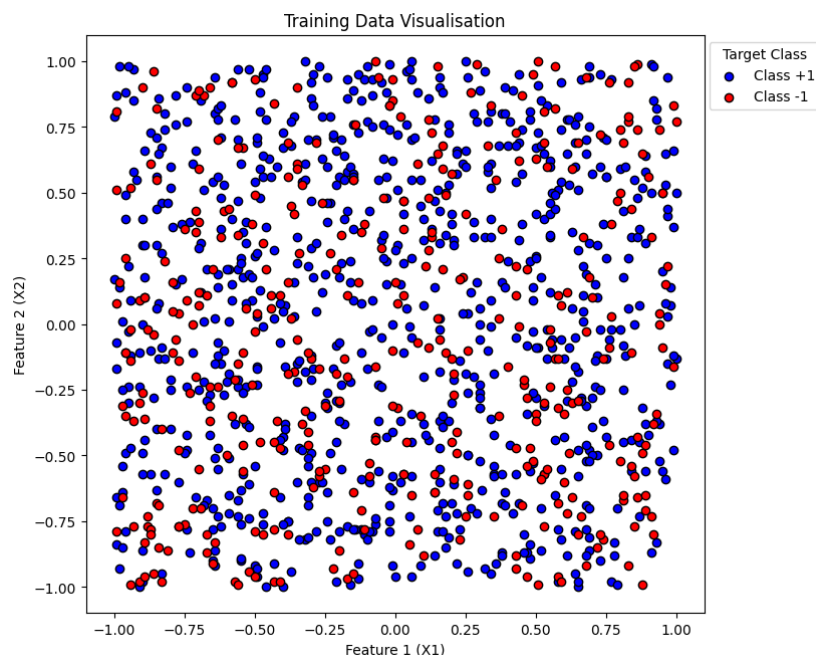


Figure 1. Visualisation of Training Data (Dataset 1)

- (a) The two input features in the dataset were augmented with polynomial features of degrees 2, 4, and 6 to train a logistic regression classifier with an L_2 penalty added to the cost function. For each polynomial degree, the model was trained across a range of C values [0.001, 0.01, 0.1, 1, 10, 100, 1000], which control the strength of the penalty (smaller values of C imply a stronger penalty on the cost function). Model performance was evaluated using 5-fold cross-validation to identify the optimal combination of polynomial degree and C value.

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score, KFold

degrees = [2, 4, 6]
C_vals = [0.001, 0.01, 0.1, 1, 10, 100, 1000]
kf = KFold(n_splits=5, shuffle=True, random_state=42)
```

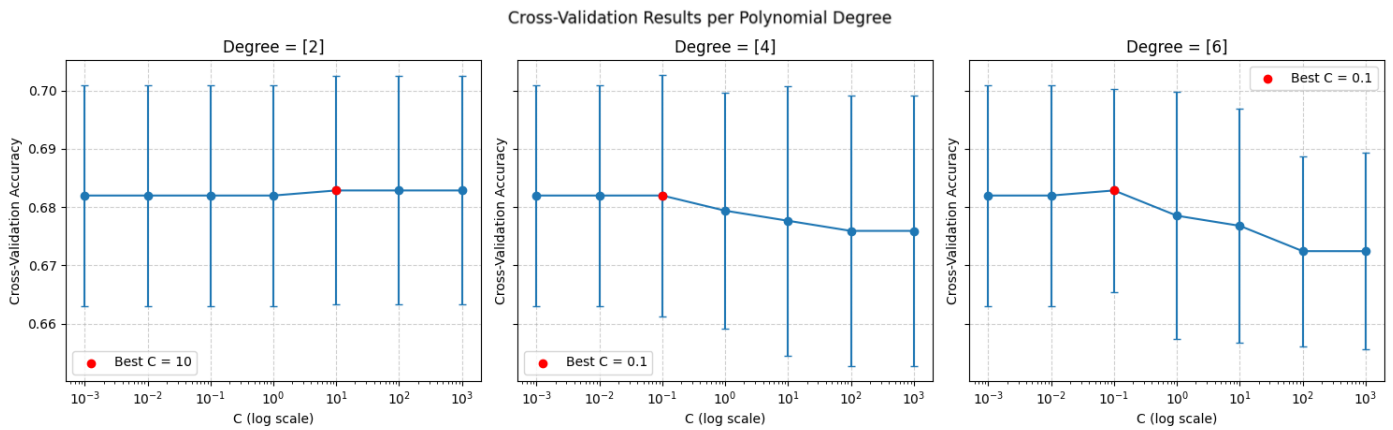


Figure 2. Plot of C values vs Cross Validation Accuracy per Polynomial Degree (Dataset 1)

Figure 2 shows the cross-validation accuracy for each polynomial degree over the range of C values. The margin of error for cross-validation accuracy in models of all degrees is large. The models with degrees 4 and 6 both achieved their highest mean accuracies (0.6819 and 0.6828, respectively) at $C = 0.1$, before their performance declined as C increased. In contrast, the degree-2 model reached its highest accuracy (0.6828) at $C = 10$, where the accuracy stabilised and remained consistent for larger values of C. Based on this balance between accuracy and stability, degree 2 and $C = 10$ were selected as the optimal parameters for the model. Using these optimal hyperparameters, the logistic regression classifier was then retrained, and its predictions were visualised alongside the decision boundary, as shown in Figure 3.

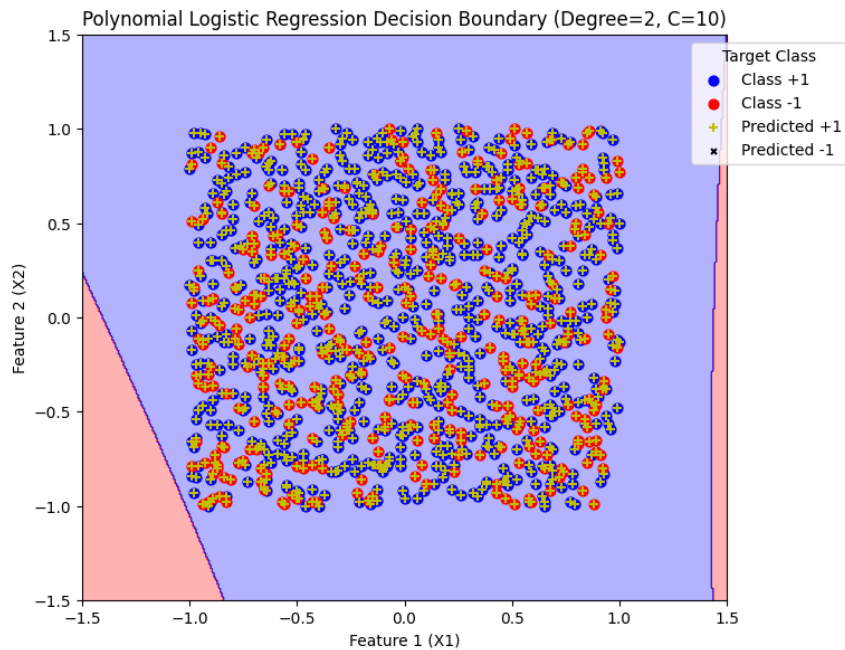


Figure 3. Logistic Regression - Predicted Classes & Decision Boundary (Dataset 1)

The model predicted all training samples as belonging to the positive class ($y = +1$), and the resulting decision boundary did not intersect any data points. As previously mentioned, with both positive and negative classes being heavily intermixed, the dataset is not linearly separable; therefore, it was to be expected that no clear boundary (linear or nonlinear) could effectively separate the two target classes.

- (b) The two input features from the dataset were next used to train a k-Nearest Neighbours (kNN) classifier. Likewise with the logistic regression model, 5-fold cross-validation was used to determine the optimal number of neighbours, k , by selecting the value that achieved the highest mean cross-validation accuracy. A range of k values from 1 to 44 was tested.

```

from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_score, KFold

k_values = np.arange(1, 45)
mean_scores = []
std_scores = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(knn, X, y, cv=kf, scoring='accuracy')
    mean_scores.append(scores.mean())
    std_scores.append(scores.std())

mean_scores = np.array(mean_scores)
std_scores = np.array(std_scores)

```

In doing this, a plot of the cross-validation accuracies for a range of k values could be produced:

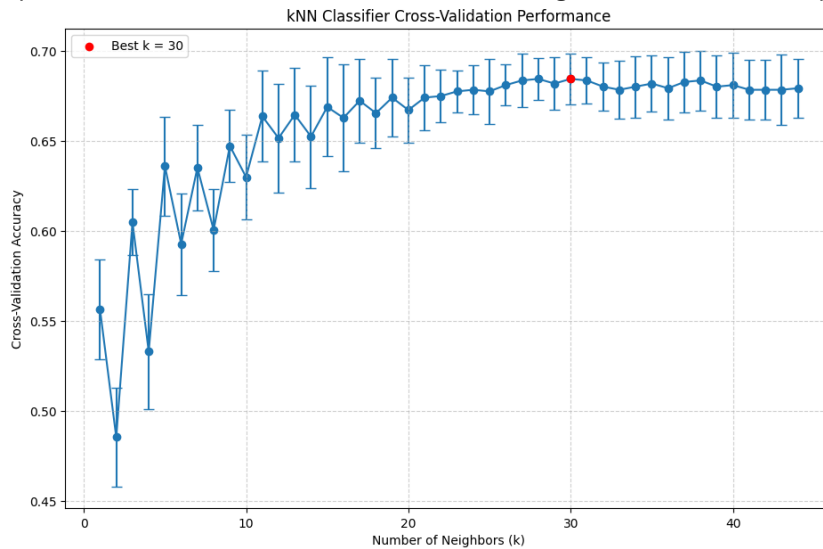


Figure 4. Plot of Cross Validation Accuracy for a range of k values (Dataset 1)

Figure 4 shows the mean cross-validation accuracy plotted against the number of neighbours, k . The results indicate that model accuracy generally increases as k increases, while variance (error) decreases correspondingly. This upward trend in performance appears to plateau near $k=30$, where the model achieves its maximum cross-validation accuracy of 0.6846. Beyond this point, the accuracy fluctuates slightly but remains near, though slightly below, its peak. Therefore, $k=30$ was selected as the optimal number of neighbours for the model.

With the optimal value of k determined, the kNN classifier was retrained on the full dataset, and its predictions were visualised alongside the resulting decision boundary, as shown in Figure 5.

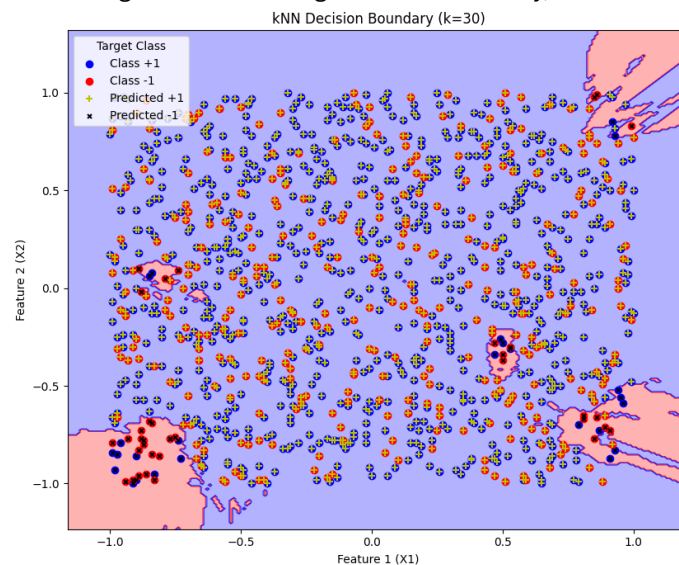


Figure 5. kNN - Predicted Classes & Decision Boundary (Dataset 1)

Although the decision boundary intersects the data region at certain edges and isolated pockets, resulting in a small number of samples being correctly classified as belonging to the negative class ($y = -1$), most of the predictions again belong to the positive class ($y = +1$). This behaviour suggests that the model is biased toward the dominant class, reflecting the class imbalance present in the dataset and the difficulty of separating the two target classes based on their overlapping feature distributions.

- (c) To further evaluate model performance, confusion matrices were constructed for the trained logistic regression and kNN classifiers. The `confusion_matrix` function from `sklearn.metrics` was used to calculate the confusion matrices, with a custom helper function, `def confuseMtrx_df()`, implemented to format the results neatly as `pd.DataFrame` tables.

```
import pandas as pd
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

def confuseMtrx_df(y_true, y_pred, labels=[1, -1]):
    cm = confusion_matrix(y_true, y_pred, labels=labels)
    return pd.DataFrame(cm,
                        index=[f"True {l}" for l in labels],
                        columns=[f"Pred {l}" for l in labels])
```

Which would produce a confusion matrix in the following format:

	Confusion Matrix	
	Pred +1	Pred -1
True +1	TP	FN
True -1	FP	TN

Table 1. Example Confusion Matrix

where TP and TN are the counts of true positive and true negative predictions, and FP and FN are the counts of false positive and false negative predictions, respectively. The accuracy of the confusion matrix could then be computed using the standard formula:

$$Accuracy_{CM} = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}} = \frac{TP + TN}{TP + TN + FP + FN}$$

Logistic Regression & kNN Classifier Confusion Matrices:

```
y_pred_logreg = model.predict(X_poly)
y_pred_knn = knn_best.predict(X)

def confuseMtrx_df(y_true, y_pred, labels=[1, -1]):
    (...)

cm_logreg_df = confuseMtrx_df(y, y_pred_logreg)
cm_knn_df = confuseMtrx_df(y, y_pred_knn)
```

N_{+1} : 787 N_{-1} : 367	Logistic Regression [dg=2, C=10]		kNN (k=30)	
	Pred +1	Pred -1	Pred +1	Pred -1
True +1	787	0	760	27
True -1	367	0	332	35
Accuracy	0.6820		0.6889	

Table 2. Logistic Regression & kNN Classifier Confusion Matrices + Accuracy Values (Dataset 1)

Both models show a heavy bias towards predicting the positive class (+1), with the logistic regression classifier failing to classify any negative samples correctly. The kNN classifier performs slightly better, correctly identifying a small number of negative samples (35) while maintaining similar accuracy on the positive class. This reflects the dataset's non-linear, overlapping structure and class imbalance, as previously discussed in (a).

Two baseline models were also implemented using the `DummyClassifier` function from `sklearn.dummy`; one which predicted the most frequent class in the dataset and another that predicted classes at random. From these baseline models two more confusion matrices were produced.

Baseline Confusion Matrices:

```
from sklearn.dummy import DummyClassifier

baseline_mostfreq = DummyClassifier(strategy='most_frequent').fit(X, y)
y_pred_mostfreq = baseline_mostfreq.predict(X)

baseline_random = DummyClassifier(strategy='uniform', random_state=42).fit(X, y)
y_pred_random = baseline_random.predict(X)

def confuseMtrx_df(y_true, y_pred, labels=[1, -1]):
    (...)

cm_mostfreq_df = confuseMtrx_df(y, y_pred_mostfreq)
cm_random_df = confuseMtrx_df(y, y_pred_random)
```

N_{+1} : 787 N_{-1} : 367	Baseline (Most Frequent)		Baseline (Random)	
	Pred +1	Pred -1	Pred +1	Pred -1
True +1	787	0	406	381
True -1	367	0	171	196
Accuracy	0.6820		0.5217	

Table 3. Baseline Confusion Matrices + Accuracy Values (Dataset 1)

The most frequent baseline achieves the same behaviour as the logistic regression classifier, predicting all samples as belonging to the positive class, indicating no meaningful discrimination between the two classes. The random baseline produces a more balanced distribution of predictions, as expected, but with lower overall accuracy.

These comparisons confirm that neither the logistic regression nor *kNN* classifiers can capture a decision boundary capable of effectively distinguishing between the two target classes, reinforcing earlier observations that the dataset's classes are not easily separable due to high overlap in feature space and class imbalance.

- (d) To provide a more detailed comparison of classifier performance beyond accuracy and confusion matrices, Receiver Operating Characteristic (ROC) curves were generated for the trained logistic regression and *kNN* classifiers, as well as for both baseline models. The `roc_curve` and `auc` functions from `sklearn.metrics` were used to compute the required metrics, as shown below.

```
from sklearn.metrics import roc_curve, auc

y_score_logreg = model.predict_proba(X_poly)[:, 1]
y_score_knn = knn_best.predict_proba(X)[:, 1]
y_score_mostfreq = baseline_mostfreq.predict_proba(X)[:, 1]
y_score_random = baseline_random.predict_proba(X)[:, 1]

fpr_logreg, tpr_logreg, _ = roc_curve(y, y_score_logreg, pos_label=1)
roc_auc_logreg = auc(fpr_logreg, tpr_logreg)
fpr_knn, tpr_knn, _ = roc_curve(y, y_score_knn, pos_label=1)
roc_auc_knn = auc(fpr_knn, tpr_knn)

fpr_mostfreq, tpr_mostfreq, _ = roc_curve(y, y_score_mostfreq, pos_label=1)
fpr_random, tpr_random, _ = roc_curve(y, y_score_random, pos_label=1)
```

The ROC curve illustrates the trade-off between the True Positive Rate (TPR) and False Positive Rate (FPR) across various classification thresholds. The Area Under the Curve (AUC) value provides a single metric to summarise overall classifier performance, where an AUC of 0.5 corresponds to random guessing, and values closer to 1.0 indicate better discernment between classes.

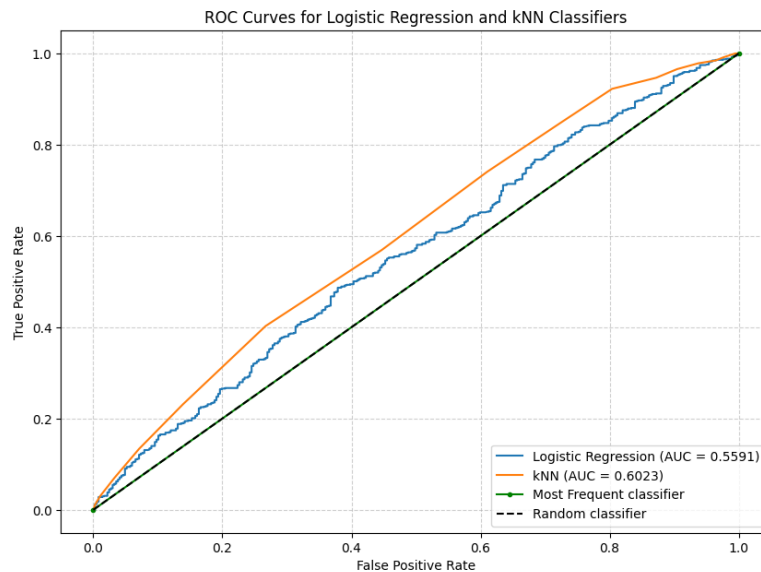


Figure 6. ROC curves for various classifiers (Dataset 1)

Figure 6 displays the ROC curves for all four classifiers. The logistic regression classifier achieved an AUC of 0.56, while the kNN classifier performed slightly better with an AUC of 0.60, indicating only a modest discriminative capability. The most frequent and random baseline classifiers both achieved an AUC of 0.5, corresponding to the chance-level performance, the 45° diagonal line that occupies the ROC space.

- (e) Comparing classifiers using previously generated confusion matrices and ROC curves shows that kNN (@ $k = 30$) performs slightly better than logistic regression and both baseline classifiers. kNN achieves a higher confusion matrix accuracy of 0.689 and an AUC of 0.60 and is the only model to correctly predict a small but significant number of negative classes. Logistic regression collapsed to predicting the majority class and matches the most frequent baseline with a confusion matrix accuracy of 0.682 and an AUC of 0.56. Although kNN is the recommended model here, the improvement from the logistic regression is minor. The dataset innately exhibits strong class overlap, meaning that simply using a better classifier would not improve prediction accuracy. Instead, significant improvements in predictions will probably require better features, different modelling approaches, such as ensemble or kernel methods, or class rebalancing.

Part II - Dataset 2:

The same machine learning workflow from part (i), encompassing feature augmentation, model training, cross-validation, and evaluation, was repeated for the second dataset, which shares the same structure as the first. For brevity, the implementation code is not repeated here since the methodology and parameters remain identical, with only the dataset changed.

Figure 7 presents a two-dimensional visualisation of the input features and corresponding ± 1 target values of this second training dataset. Unlike the first dataset, this data exhibits a clearly non-linear decision boundary, where points belonging to the positive class (+1) form an inverted parabolic cluster, surrounded by negative class (-1) points. This pattern suggests that a linear model will not effectively separate the two classes, but either a polynomial logistic regression model or a non-linear classifier such as kNN may capture the boundary more accurately.

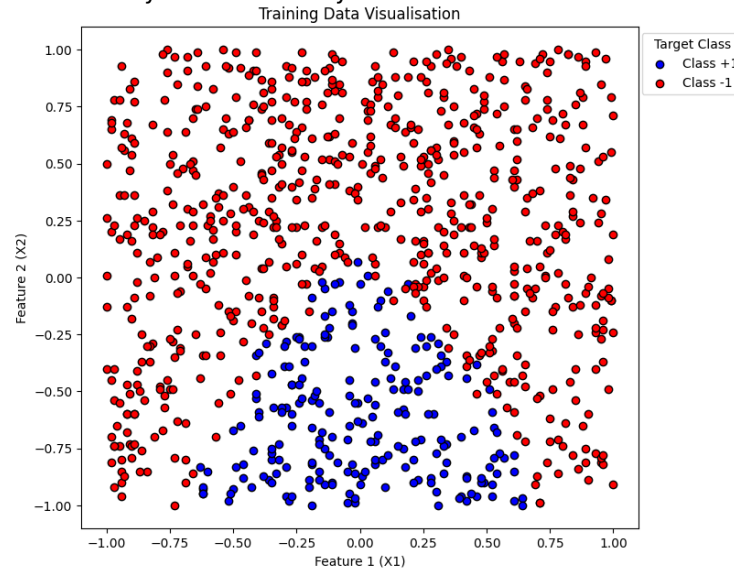


Figure 7. Visualisation of Training Data (Dataset 2)

- (a) The two features in the second dataset were augmented with polynomial features spanning degrees [2, 4, and 6], and a logistic regression classifier with an L_2 penalty was trained using the same range of C values tested previously [0.001, 0.01, 0.1, 1, 10, 100, 1000]. Five-fold cross-validation was again used to determine the optimal polynomial degree and penalty weight.

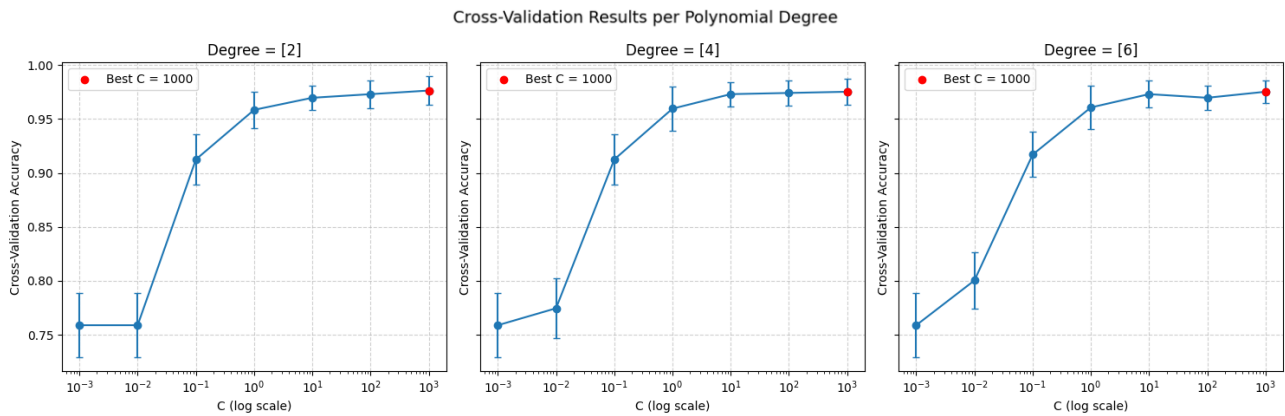


Figure 8. Plot of C values vs Cross Validation Accuracy per Polynomial Degree (Dataset 2)

The cross-validation results in Figure 8 show that model performance improves substantially as the polynomial degree and regularisation weight increase. The model achieves its highest cross-validation accuracy of 0.9764 at degree = 2 and $C = 1000$, after which performance plateaus. Increasing the polynomial degree beyond 2 yields no notable improvement and, in some cases, slight overfitting effects. With these optimal parameters, the logistic regression model was retrained on the full dataset.

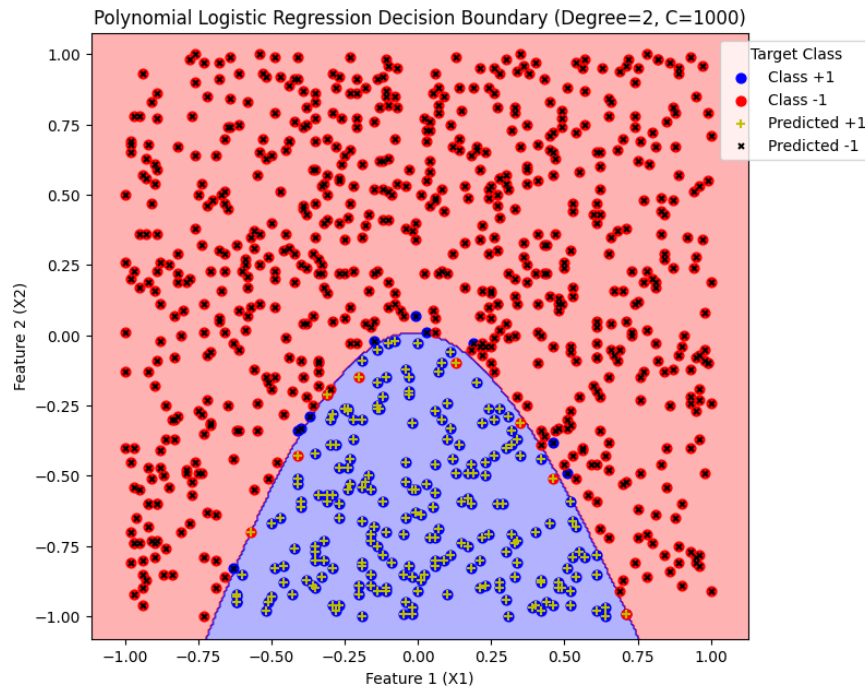


Figure 9. Logistic Regression - Predicted Classes & Decision Boundary (Dataset 2)

As shown in Figure 9, the model correctly classifies almost all samples, with only a few misclassifications occurring near the region where the two classes overlap. The decision boundary follows the inverted parabola shape seen in the training data, effectively separating the positive and negative classes. Compared to the first dataset, this second dataset is well captured by a non-linear boundary.

- (b) To train and evaluate a k NN classifier on the dataset, the same range of k values (1 to 44) was tested using 5-fold cross-validation. Figure 10 shows how the mean cross-validation accuracy scores gradually increased with k , peaking at $k = 10$ with a maximum average accuracy of 0.9731. The standard deviation across folds for this k value was at its lowest (0.00414), implying that $k=10$ has the most stable classifier performance across all the k values.

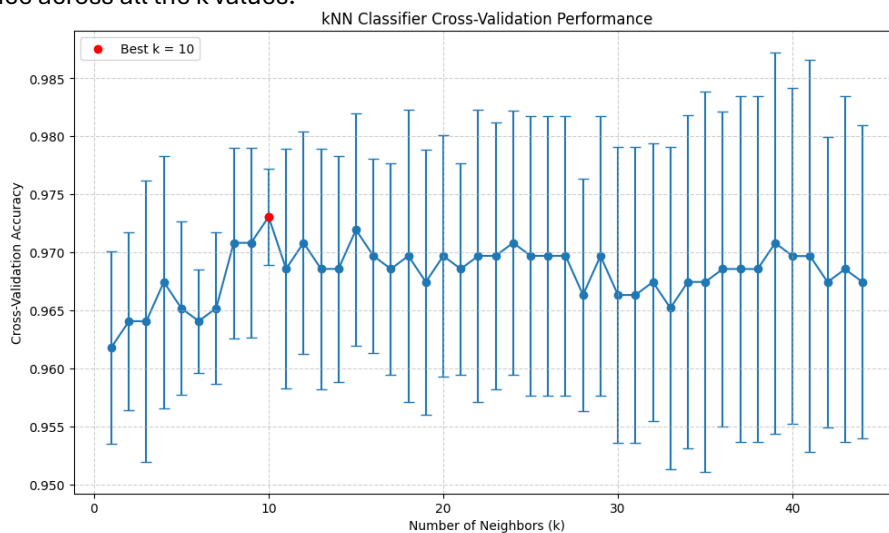


Figure 10. Plot of Cross Validation Accuracy for a range of k values (Dataset 2)

The kNN classifier, now trained with the optimal parameter ($k = 10$), correctly classified almost all samples, as both target classes (-1 and $+1$) were represented in the predictions, as can be observed in figure 11. Similar to the logistic regression model, a small number of misclassifications occurred in regions where the two classes overlap. The resulting decision boundary effectively traced the non-linear separation between the classes, following the inverted parabolic region observed in the data. However, unlike logistic regression's smoother curve, the kNN boundary appeared more irregular and jagged.

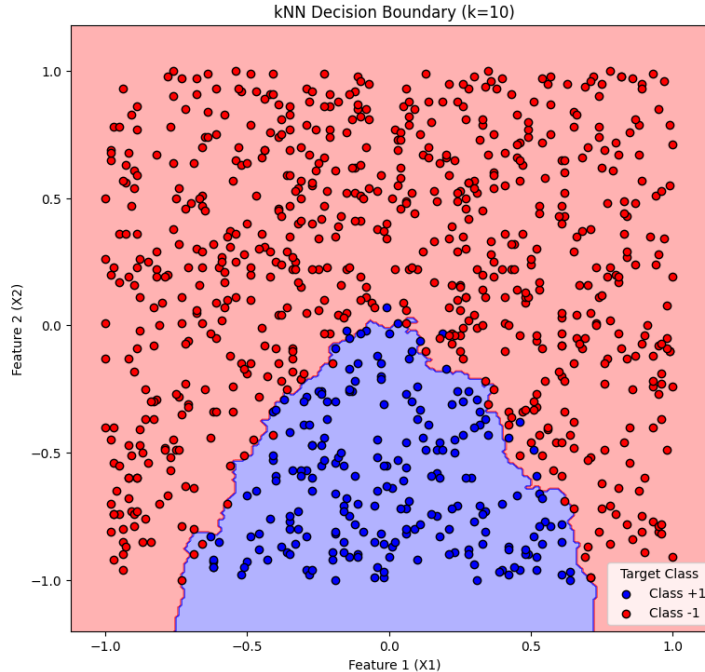


Figure 11. kNN - Predicted Classes & Decision Boundary (Dataset 2)

- (c) The confusion matrices for the logistic regression model, kNN classifier, and the two baseline classifiers are shown in the tables below. These reflect how well each model predicted the two target classes using the second dataset.

Logistic Regression & kNN Classifier Confusion Matrices:

N_{+1} : 215 N_{-1} : 676	Logistic Regression [dg=2, C=10]		kNN Classifier ($k=30$)	
	Pred +1	Pred -1	Pred +1	Pred -1
True +1	205	10	202	13
True -1	8	668	7	669
Accuracy	0.9798		0.9776	

Table 4. Logistic Regression & kNN Classifier Confusion Matrix + Accuracy Values (Dataset 2)

The logistic regression classifier achieved strong performance, correctly classifying the majority of both positive and negative samples with only a small number of misclassifications near the class overlap region. The kNN classifier performed comparably, showing a very similar confusion pattern and slightly fewer misclassifications of negative instances but also slightly more misclassifications of positive values.

Baseline Confusion Matrices:

N_{+1} : 215 N_{-1} : 676	Baseline (Most Frequent)		Baseline (Random)	
	Pred +1	Pred -1	Pred +1	Pred -1
True +1	0	215	114	101
True -1	0	676	339	337
Accuracy	0.7587		0.5061	

Table 5. Baseline Confusion Matrices + Accuracy Values (Dataset 2)

In contrast, the baseline classifiers performed expectedly worse. The “most frequent” model ignored the minority class, and the “random” classifier’s predictions were self-evidently random, producing large misclassification counts. These results demonstrate that both the logistic regression and kNN models substantially outperform the baseline approaches, with only minor differences between each other.

- (d) The Receiver Operating Characteristic (ROC) curves and corresponding Area Under the Curve (AUC) values were computed for the optimal logistic regression (@degree = 2, $C = 1000$), kNN (@ $k = 10$), and both baseline classifiers, seen in Figure 12.

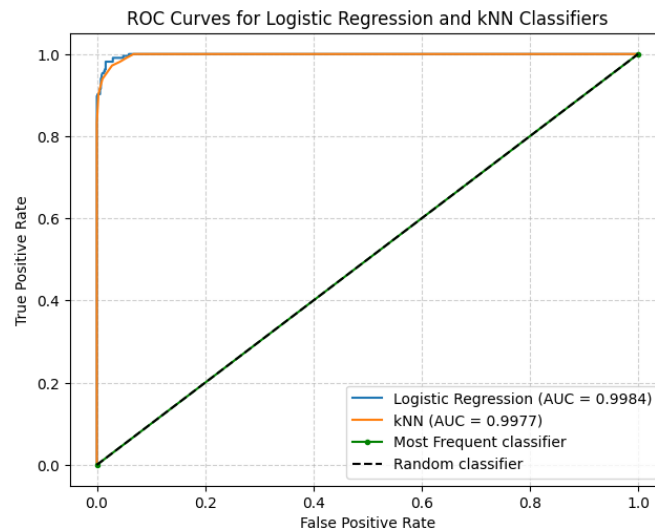


Figure 12. ROC curves for various classifiers (Dataset 2)

Both the logistic regression and kNN classifiers achieve near perfect ROC curves, indicating exceptional discriminative ability between the positive and negative classes. The curves approach the top-left corner of the ROC space, reflecting high true positive rates and very low false positive rates. In contrast, the baseline classifiers lie along the diagonal of the ROC space, consistent with their random or most frequent prediction strategies. These AUC values reinforce the confusion matrix results from part (ii)(c), showing that both logistic regression and kNN far outperform baseline models, with only a minor difference between the two.

- (e) Comparing classifiers using the confusion matrices and ROC curves shows that both logistic regression and kNN perform exceptionally well on this dataset, far outperforming the baseline models. While the kNN classifier (@ $k = 10$) achieves a confusion matrix accuracy of 0.9776 and an AUC of 0.9977, the logistic regression model (@degree = 2, $C = 1000$) slightly outperforms kNN in confusion matrix accuracy, with 0.9798 and AUC = 0.9984. The baseline classifiers perform substantially worse: the “most frequent” classifier achieves 0.7587 confusion matrix accuracy and the random classifier 0.5061, with both having an AUC of 0.5 as expected.

Given these results, logistic regression is recommended for this dataset due to its slightly higher accuracy, smoother decision boundary, and near perfect discriminative performance. Even so, kNN remains a viable option, particularly if a non-parametric approach is preferred, though its improvement over logistic regression is minimal.

Appendix

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from matplotlib.colors import ListedColormap
from sklearn.model_selection import cross_val_score, KFold

file = pd.read_csv('week4.1.csv', header=None)      #(i) - dataset1
#file = pd.read_csv('week4.2.csv', header=None)      #(ii) - dataset2

X = file.iloc[:, 0:2].values #col1(X[0]->feat1)=x-axis, col2(X[1]->feat2)=y-axis,
y = file.iloc[:, 2].values   #col3(X[2]->target(y))

#:::(for(a)&(b)):::
#X vals separated for y predictions
X1 = file.iloc[:,0]
X2 = file.iloc[:,1]
#5 folds for cross validation
kf = KFold(n_splits=5, shuffle=True, random_state=42)
#meshgrid for better dec boundary visual
x1_min, x1_max = X1.min() - 0.5, X1.max() + 0.5
x2_min, x2_max = X2.min() - 0.5, X2.max() + 0.5
xx, yy = np.meshgrid(np.linspace(x1_min, x1_max, 400),
                     np.linspace(x2_min, x2_max, 400))
#::::::::::::::::::::

#training data plot
plt.figure(figsize=(6,6))
#training data
plt.scatter(X[y == 1, 0], X[y == 1, 1], color='blue', edgecolor='k', label='Class +1')
plt.scatter(X[y == -1, 0], X[y == -1, 1], color='red', edgecolor='k', label='Class -1')
#lables & legend
plt.xlabel("Feature 1 (X1)")
plt.ylabel("Feature 2 (X2)")
plt.title("Training Data Visualisation")
plt.legend(title="Target Class", loc='upper right', bbox_to_anchor=(1.20, 1))
plt.tight_layout()
plt.show()
#see count of each train data class(for(c))
print(np.unique(y, return_counts=True))

'''
#------(a)-----
'''
#---polynomial logistic regression---
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LogisticRegression

degrees = [2, 4, 6]
C_vals = [0.001, 0.01, 0.1, 1, 10, 100, 1000]

mean_scores = np.zeros((len(degrees), len(C_vals)))
std_scores = np.zeros((len(degrees), len(C_vals)))

for i, d in enumerate(degrees):
    poly = PolynomialFeatures(degree=d)
    X_poly = poly.fit_transform(X)

    for j, c in enumerate(C_vals):
        #train log reg
        model = LogisticRegression(C=c, penalty='l2', solver='lbfgs', max_iter=10000)
        #5F cross val
        scores = cross_val_score(model, X_poly, y, cv=kf, scoring='accuracy')
        mean_scores[i, j] = scores.mean() #cross validation accuracy
        std_scores[i, j] = scores.std()   #accuracy error

#print(mean_scores.round(5))

fig, axes = plt.subplots(1, 3, figsize=(15, 4), sharey=True) # 1x3 grid for 3 degree subplots
axes = axes.ravel() #flatten to index with [i]

for i, d in enumerate(degrees):
```

```

ax = axes[i]
    #for std.dev of cross val accuracy
ax.errorbar(C_vals, mean_scores[i], yerr=std_scores[i], fmt='--o', capsize=3)
    #labels
ax.set_xscale('log')
ax.set_title(f"Degree = [{d}]")
ax.set_xlabel("C (log scale)")
ax.set_ylabel("Cross-Validation Accuracy")
ax.grid(True, linestyle='--', alpha=0.6)
    #find max (best) accuracy for each degree
best_idx = np.argmax(mean_scores[i])
print(f"Degree = [{d}]      C = {C_vals[best_idx]}")
print(f"CV Accuracy: {mean_scores[i, best_idx].round(5)}")
    #use to find+highlight best C val in plot
ax.scatter(C_vals[best_idx], mean_scores[i, best_idx], color='red', zorder=5, label=f"Best C = {C_vals[best_idx]}")
ax.legend()

#auto pick best deg & C val out of all plots made
best_deg_idx, best_c_idx = np.unravel_index(np.argmax(mean_scores), mean_scores.shape)
best_degree = degrees[best_deg_idx]
best_C = C_vals[best_c_idx]
print("best degree :", best_degree)
print("best C :", best_C)

plt.suptitle("Cross-Validation Results per Polynomial Degree", y=1.0, fontsize=14)
plt.tight_layout()
plt.show()

---BEST polynom log reg---
poly = PolynomialFeatures(degree=best_degree)
X_poly = poly.fit_transform(X)
    #train log reg
model = LogisticRegression(C=best_C, penalty='l2', solver='lbfgs', max_iter=10000)
model.fit(X_poly, y)
y_pred = model.predict(X_poly)

#plot
plt.figure(figsize=(7,7))
    #training data
plt.scatter(X1[y == 1], X2[y == 1], s=40, marker='o', color='b', zorder=5, label='Class +1')
plt.scatter(X1[y == -1], X2[y == -1], s=40, marker='o', color='r', zorder=5, label='Class -1')
    #predictions
plt.scatter(X1[y_pred == 1], X2[y_pred == 1], marker='+', color='y', zorder=5, label='Predicted +1')
plt.scatter(X1[y_pred == -1], X2[y_pred == -1], s=12.5, marker='x', color='k', zorder=5, label='Predicted -1')
    #prediction regions on meshgrid
Z = model.predict(poly.transform(np.c_[xx.ravel(), yy.ravel()]))
Z = Z.reshape(xx.shape)
    #dec boundary
plt.contourf(xx, yy, Z, alpha=0.3, cmap=ListedColormap(['red', 'blue']))
    #labels&legend
plt.xlabel("Feature 1 (X1)")
plt.ylabel("Feature 2 (X2)")
plt.title(f"Polynomial Logistic Regression Decision Boundary (Degree={best_degree}, C={best_C})")
plt.legend(title="Target Class", loc='upper right', bbox_to_anchor=(1.20, 1))

plt.tight_layout()
plt.show()
'''
#------(b)-----
'''
#-----kNN-----
from sklearn.neighbors import KNeighborsClassifier

#range of k values to test
k_values = np.arange(1, 45)
#reset acc & err
mean_scores = []
std_scores = []

for k in k_values:
    #train kNN
    knn = KNeighborsClassifier(n_neighbors=k)

```

```

    #5F cross val
    scores = cross_val_score(knn, X, y, cv=kf, scoring='accuracy')
    mean_scores.append(scores.mean())
    std_scores.append(scores.std())

mean_scores = np.array(mean_scores)
std_scores = np.array(std_scores)

#print(mean_scores.round(5))
#print(std_scores.round(5))

#plot
plt.figure(figsize=(8,5))
    #for std.dev of cross val accuracy
plt.errorbar(k_values, mean_scores, yerr=std_scores, fmt='-o', capsize=4, zorder=1)
    #lables
plt.xlabel("Number of Neighbors (k)")
plt.ylabel("Cross-Validation Accuracy")
plt.title("kNN Classifier Cross-Validation Performance")
plt.grid(True, linestyle='--', alpha=0.6)
    #find max (best) accuracy for each k val
best_k = k_values[np.argmax(mean_scores)]
print(f"Best k = {best_k}, Mean CV Accuracy = {np.max(mean_scores):.4f}")
    #use to find+highlight best k in plot
plt.scatter(best_k, np.max(mean_scores), color='red', zorder=2, label=f"Best k = {best_k}")
plt.legend()
plt.tight_layout()
plt.show()

#-----BEST kNN-----
    #train kNN
knn_best = KNeighborsClassifier(n_neighbors=best_k).fit(X, y)
y_pred_knn = knn_best.predict(X)
    #prediction regions on meshgrid
Z = knn_best.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

#plot
plt.figure(figsize=(7,7))
    #dec boundary
plt.contourf(xx, yy, Z, alpha=0.3, cmap=ListedColormap(['red', 'blue']))
    #training data
plt.scatter(X[y==1,0], X[y==1,1], color='blue', label='Class +1')
plt.scatter(X[y==1,0], X[y==1,1], color='red', label='Class -1')
    #predictions
plt.scatter(X1[y_pred_knn == 1], X2[y_pred_knn == 1], marker='+', color='y', zorder=5, label='Predicted +1')
plt.scatter(X1[y_pred_knn == -1], X2[y_pred_knn == -1], s=12.5, marker='x', color='k', zorder=5,
label='Predicted -1')
    #legend&lables
plt.title(f"kNN Decision Boundary (k={best_k})")
plt.xlabel("Feature 1 (X1)")
plt.ylabel("Feature 2 (X2)")
plt.legend(title="Target Class", loc='upper left')

plt.tight_layout()
plt.show()
'''
#------(c)-----
'''
#-----confusion matrix-----
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from sklearn.dummy import DummyClassifier

#polynom Log Reg predictions
y_pred_logreg = model.predict(X_poly)
    #see count of each data class
print(np.unique(y_pred_logreg, return_counts=True))

#kNN predictions
y_pred_knn = knn_best.predict(X)
    #see count of each data class
print(np.unique(y_pred_knn, return_counts=True))

```

```

#baseline models
#most frequent
baseline_mostfreq = DummyClassifier(strategy='most_frequent').fit(X, y)
y_pred_mostfreq = baseline_mostfreq.predict(X)
#random
baseline_random = DummyClassifier(strategy='uniform', random_state=42).fit(X, y)
y_pred_random = baseline_random.predict(X)

#func to create labeled confusion matrix
def confuseMtrx_df(y_true, y_pred, labels=[1, -1]):
    cm = confusion_matrix(y_true, y_pred, labels=labels)
    return pd.DataFrame(
        cm,
        index=[f"True {l}" for l in labels],
        columns=[f"Pred {l}" for l in labels]
    )

#build+print dataframes
#pynm log reg
cm_logreg_df = confuseMtrx_df(y, y_pred_logreg)
print("\n--- Confusion Matrix: Logistic Regression ---")
print(cm_logreg_df)
#kNN
cm_knn_df = confuseMtrx_df(y, y_pred_knn)
print(f"\n--- Confusion Matrix: kNN (k={best_k}) ---")
print(cm_knn_df)
#freq
cm_mostfreq_df = confuseMtrx_df(y, y_pred_mostfreq)
print("\n--- Confusion Matrix: Baseline (Most Frequent) ---")
print(cm_mostfreq_df)
#rand
cm_random_df = confuseMtrx_df(y, y_pred_random)
print("\n--- Confusion Matrix: Baseline (Random) ---")
print(cm_random_df)

'''
#------(d)-----
'''
#-----ROC Curve-----
from sklearn.metrics import roc_curve, auc
# pynm log reg probs
#(.predict(...)-> get class prediction|||.predict_proba(...)-> get prob value)
y_score_logreg = model.predict_proba(X_poly)[: , 1]

# kNN probs
y_score_knn = knn_best.predict_proba(X)[: , 1]

# baseln probs
y_score_mostfreq = baseline_mostfreq.predict_proba(X)[: , 1]
y_score_random = baseline_random.predict_proba(X)[: , 1]

#get false pos & true pos rates - logreg
fpr_logreg, tpr_logreg, _ = roc_curve(y, y_score_logreg, pos_label=1)
#use to find AUC
roc_auc_logreg = auc(fpr_logreg, tpr_logreg)

#get false pos & true pos rates - kNN
fpr_knn, tpr_knn, _ = roc_curve(y, y_score_knn, pos_label=1)
roc_auc_knn = auc(fpr_knn, tpr_knn)

#get false pos & true pos rates - baselns
fpr_mostfreq, tpr_mostfreq, _ = roc_curve(y, y_score_mostfreq, pos_label=1)
fpr_random, tpr_random, _ = roc_curve(y, y_score_random, pos_label=1)

#plot
plt.figure(figsize=(7,7))
#ROC curves
plt.plot(fpr_logreg, tpr_logreg, label=f"Logistic Regression (AUC = {roc_auc_logreg:.4f})")
plt.plot(fpr_knn, tpr_knn, label=f"kNN (AUC = {roc_auc_knn:.4f})")
#freq baseln
plt.plot(fpr_mostfreq, tpr_mostfreq, c='g', marker='.', label='Most Frequent classifier')
#diagonal (rand Baseln)
plt.plot(fpr_random, tpr_random, 'k--', label='Random classifier')

```

```
        #lables&legend
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curves for Logistic Regression and kNN Classifiers")
plt.legend(loc="lower right")
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
#'''
```